

Paper review: STSR by Zhao et al.

Presented by Syed Ukkashah Ahmed Shah (23K-0055)

STSR

Problem statement

Teams of cooperating agents learn faster when they all share one policy network. That works for identical agents, like ten marines, but breaks down when a team mixes unit types with different inputs, outputs and jobs.

Padding everyone's inputs to one size wastes computation and makes the agents behave alike. A separate network per agent avoids that but trains slowly. The authors look for a middle ground for mixed teams.

RESEARCH QUESTIONS

- 1. Can different agent types share parameters and still specialise?**
- 2. Can one shared representation fit agents with different inputs and outputs?**
- 3. Do intrinsic rewards from that representation improve exploration?**
- 4. Does the method beat standard MARL baselines on mixed teams?**
- 5. Do trained policies transfer to larger teams without retraining?**

METHODOLOGY

Part	What the authors did	Detail
Representation	An encoder-decoder that predicts each agent's next observation and reward	The encoder only sees the agent's unit type
Policy sharing	Full sharing within a unit type, plus a shared bottom layer across types	Each type keeps its own upper layers
Intrinsic rewards	An exploration bonus from prediction error, and a reward for keeping types distinct	Both are added to the game reward
Benchmark	StarCraft II micromanagement (SMAC), all 14 maps	Mixed and single-type teams, easy to super hard
Baselines	VDN, QMIX, QPLEX, ROMA, RODE and CDS	4 random seeds, median reported
Extra tests	Ablations, embedding plots and transfer to larger maps	Transfer tested without retraining

o_i and a_i are agent i 's observation and action, a_{-i} is what the other agents did, and o'_i is the next observation.

z_i is the type embedding, f_o and f_r predict the observation and reward, and each λ sets how much a term counts.

EQUATIONS

$$L(\theta) = E[\sum_i \|f_o(z_i, o_i, a_{-i}) - o'_i\|^2 + \lambda_e \sum_i (f_r(z_i, o_i, a_{-i}) - r)^2]$$

Loss for the representation model. It predicts each agent's next observation and the team reward, so z_i learns what sets each unit type apart.

$$r^{\text{tot}} = r + \lambda_e r^e + \lambda_d r^d$$

Training reward: the game reward plus an exploration bonus r^e and a reward r^d for keeping unit types distinct.

$$Q_j = g_j(f(o, a))$$

Hard parameter sharing: all agents use the shared layer f , and each unit type j has its own head g_j .

FINDINGS

STSR did best on mixed teams and struggled on teams made of a single unit type.

It won the most on two mixed super hard maps, 3s5z vs 3s6z and MMM2, and converged faster there than the role-based methods.

On single-type maps it fell well behind: about 10% wins on corridor and almost none on 6h vs 8z, where RODE reached roughly 85 to 90%.

In the ablations, refining the shared layer with RL mattered most. Hard parameter sharing mainly sped up early training, and final win rates barely moved.

The learned embeddings split Medivacs, Marauders and Marines into clear, separate clusters.

Transfer to larger teams

A policy trained on 3s5z vs 3s6z was tested on bigger maps with no retraining. Each new agent copied the policy of its unit type.

It kept winning when both sides grew. When only the enemy grew, win rates fell fast, from 99% to 24% with two extra enemy Stalkers.

STSR is a practical recipe for mixed teams when unit types are known and each group is a moderate size.

Its main idea mostly speeds up training, and it fails on single-type teams, so the claim that it beats mainstream methods holds only for mixed teams.

GAPS AND LIMITATIONS

Grouping by unit type needs the types known in advance, the same prior knowledge the authors fault role-based methods for.

Results rest on 4 seeds, the paper shows only 6 of its 14 maps, and the ablations use the three best maps.

The claim to be first with hard parameter sharing in MARL is weak, since selective parameter sharing (Christianos et al., 2021) already grouped agents.

Some equations disagree with the text, and the transfer test has no baseline to compare against.